

LFM2.5-2.6B: повсеместное развертывание агентов

Сегодня мы выпускаем **LFM2.5-2.6B** – агентную модель, которая полностью работает на устройстве. Она достаточно компактна, чтобы поместиться в телефоне, достаточно быстра, чтобы не перегружать процессор, и достаточно функциональна, чтобы обеспечивать агентные рабочие процессы: планирование, вызов инструментов и выполнение многоэтапных задач.

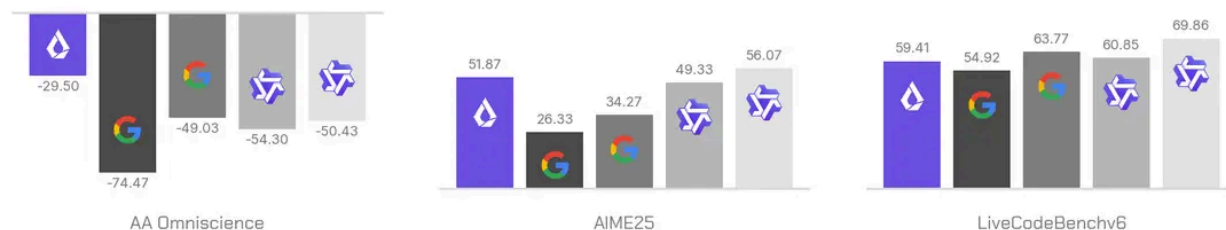
В отличие от агентов, которые зависят от облачных API, локальные агенты обеспечивают бесплатный логический вывод, низкую задержку и реальную конфиденциальность. Отсутствие платы за каждый токен меняет подход разработчиков к созданию приложений: теперь агенты можно массово распараллеливать на локальном оборудовании, выполняя фоновые задачи, которые расходуют миллионы токенов без дополнительных затрат. Когда расход токенов перестает быть сдерживающим фактором, агенты можно запускать где угодно и круглосуточно.

Базовая (LFM2.5-2.6B-Base) и дообработанная (LFM2.5-2.6B) модели доступны уже сегодня на [Hugging Face](#). Ознакомьтесь с нашими [документами](#), чтобы узнать, как запускать и дообучать их локально.

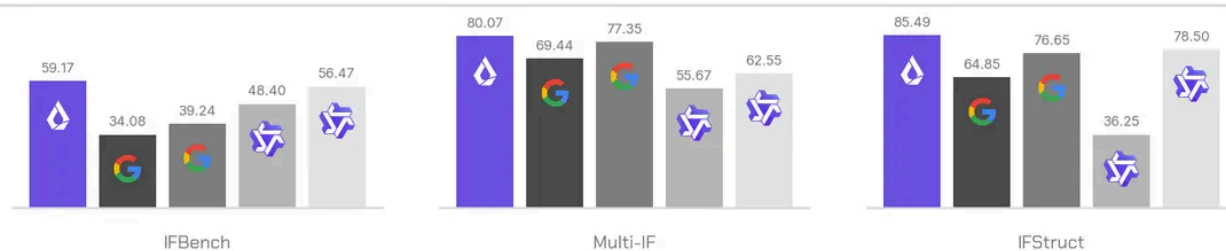
🔹 LFM2.5-2.6B Evaluations

■ LFM2.5-2.6B (2.6B) ■ gemma-4-E2B-it (5.1B) ■ gemma-4-E4B-it (8B) ■ Qwen3.5-4B (4.7B) ■ Qwen3.5-9B (9.7B)

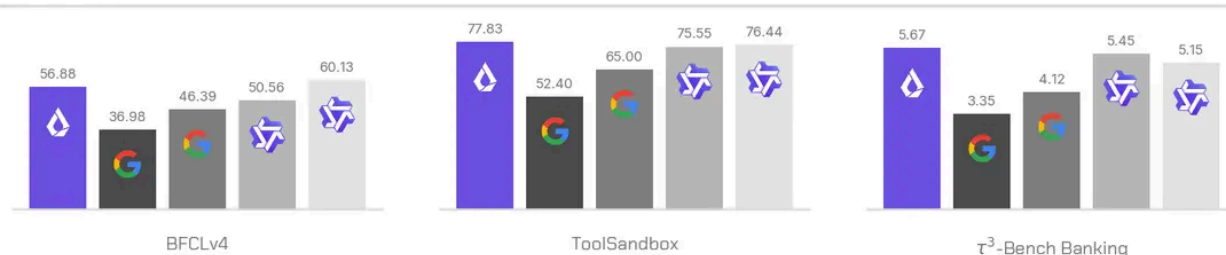
STEM



Instruction following



Tool use



Agentic



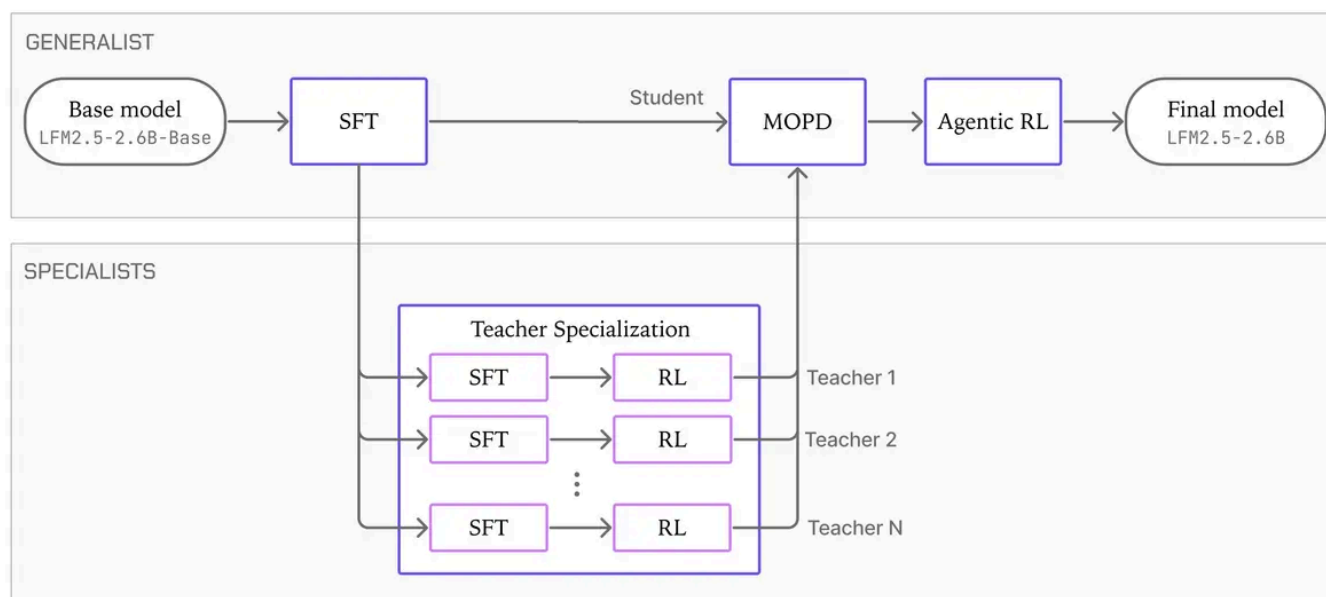
Обучение

LFM2.5-2.6B – это модель с 2,6 миллиардами параметров, специально обученная для агентских задач. Она предварительно обучена на ~34 триллионах токенов. Чтобы лучше поддерживать нелатинские алфавиты в LFM2.5, мы удвоили словарный запас до 128 тысяч, [расширив существующий токенизатор](#), а не переобучая модель с нуля, используя ту же процедуру, что и в [LFM2.5-8B-A1B](#). В середине обучения выполняется специальный этап

расширения контекста до 128 КБ, чтобы модель могла обрабатывать длинные входные данные, необходимые для агентных рабочих нагрузок.

На схеме представлен четырехэтапный процесс пост-обучения, который превращает LFM2.5-2.6B-Base в агентную модель LFM2.5-2.6B: **контролируемая дообучение** (supervised fine-tuning, SFT), **специализация на учителях**, **многодоменная дистилляция на основе политики** (multi-domain on-policy distillation, MOPD) и **агентное обучение с подкреплением** (agentic reinforcement learning, Agentic RL).

LFM2.5-2.6B Training Recipe



Тонкая настройка с учителем. Пост-обучение начинается с двух последовательных этапов тонкой настройки с учителем: сначала происходит широкий охват всех областей, а затем целенаправленное формирование приоритетных навыков, таких как агентские задачи, логическое мышление и использование инструментов. На этих двух этапах набор данных для тонкой настройки с учителем примерно в семь раз больше, чем тот, что использовался для LFM2.5-8B-A1B, и в нем больше внимания уделяется агентским задачам, таким как использование инструментов, веб-поиск, разработка программного обеспечения и трассировка агентов. Последняя контрольная точка SFT служит одновременно и обучаемой моделью, и контрольной точкой инициализации для обучения набора специализированных учителей на более позднем этапе дистилляции.

Специализация учителей. На основе общей контрольной точки SFT мы обучаем по одному эксперту в каждой целевой области с помощью целенаправленного раунда SFT на перевзвешенной выборке, за которым следует обучение с подкреплением и верифицируемыми вознаграждениями (reinforcement learning with verifiable rewards, RLVR). В результате получаются специалисты в таких областях, как следование инструкциям, математика, знания, в том числе контроль галлюцинаций, программирование, использование инструментов и работа с длинным контекстом. Раздельное обучение позволяет каждому эксперту глубоко оптимизировать свою область, используя целевые данные и вознаграждения без конкурирующих обновлений, связанных с несвязанными задачами.

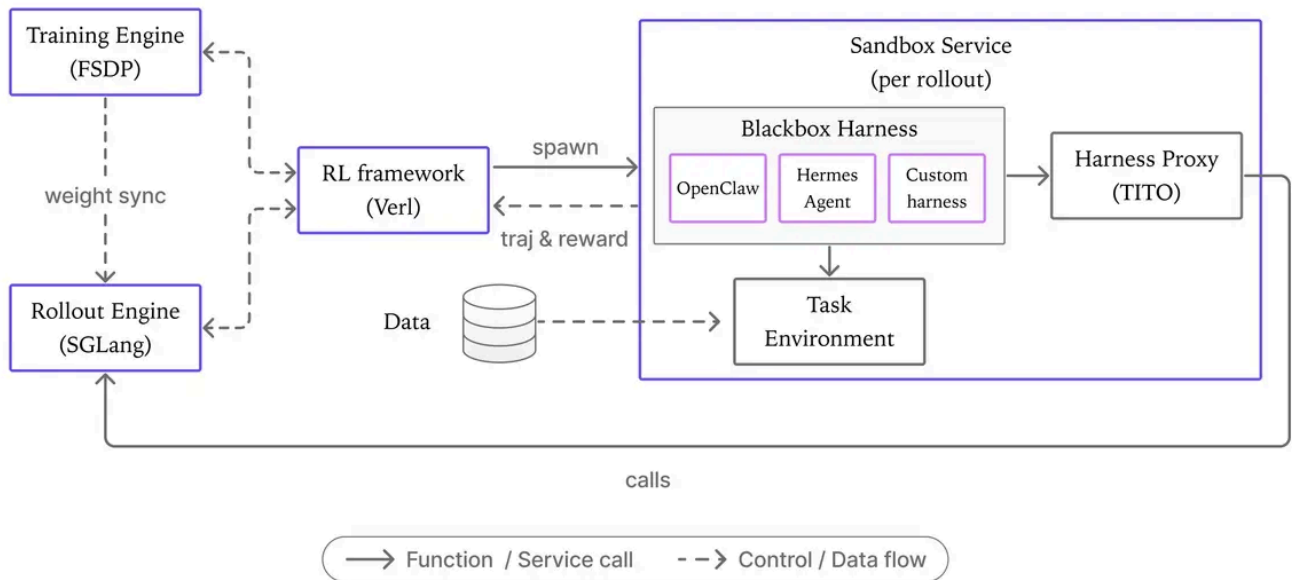
MOPD. Затем мы используем профильных экспертов в качестве наставников и объединяем их возможности в одной обучаемой модели. В отличие от дистилляции вне политики, когда обучаемая модель учится на траекториях, сгенерированных другой моделью, MOPD позволяет обучаемой модели действовать в соответствии с собственной политикой. Каждый запрос направляется к наставнику в соответствующей области, который контролирует ответ обучаемой модели, предоставляя обратную связь на уровне токенов.

Поскольку учителя начинают обучение с той же контрольной точки SFT, что и ученик, их обратная связь остается достаточно близкой к распределению ученика, что позволяет направлять процесс обучения, не нарушая его стабильность. Такое плотное направленное супервизирование помогает ученику быстро сходиться, интегрируя специализированные возможности в единую модель.

Agentic RL. The final stage teaches the model to operate inside real agent environments. We run multi-turn agentic RL through real agent harnesses, where the model works through realistic productivity tasks that evaluate its ability to research, write, code, analyze data, manage documents, use external tools, and automate multi-step workflows.

During training, we sample a task and randomly select a corresponding harness. Each rollout runs in a dedicated sandbox with its own runtime. We optimize with GRPO, using an outcome-based reward that combines an LLM-as-a-judge rubric, programmatic checks, and a hard safety gate. Training directly inside Hermes Agent, OpenClaw, and other harnesses exposes the model to their tools, system prompts, and interaction patterns, helping it work reliably across agent environments.

Agentic Reinforcement Learning



The training pipeline separates model optimization, inference, and environment execution into distinct components. The **Training Engine** (FSDP) optimizes the model, while the **Rollout Engine** (SGLang) generates actions using the latest policy. The **RL framework** (verl) orchestrates the training loop by launching rollouts, collecting trajectories and rewards, and updating the model.

Actions are executed within a **Sandbox Service**, where the **Blackbox Harness** hosts the agent (e.g., OpenClaw or Hermes Agent) and coordinates interactions with the task environment through tool calls, code execution, and other task-specific operations. The **Harness Proxy** lets us treat agentic harnesses as black boxes with no modification, while transparently capturing the token-level trajectories needed to reconstruct and validate RL training samples. This includes linear trajectory consistency, token mismatch checks, and Rollout Routing Replay (R3).

Benchmarks

We evaluated LFM2.5-2.6B across benchmarks covering STEM, instruction following, tool use, and agentic workflows. Despite being the smallest model in the comparison, it is competitive with, and often outperforms, models nearly four times its size.

Benchmark	LFM2.5- 2.6B (2.6B)	gemma- 4-E2B- it (5.1B)	gemma- 4-E4B- it (8B)	Qwen3.5- 4B (4.7B)	Qwen3.5- 9B (9.4B)
AA Omniscience	-29.50	-74.47	-49.03	-54.30	-49.00
AIME25	51.87	26.33	34.27	49.33	50.00
LiveCodeBenchv6	59.41	54.92	63.77	60.85	60.00
IFBench	59.17	34.08	39.24	48.40	50.00
Multi-IF	80.07	69.44	77.35	55.67	60.00
IFStruct	85.49	64.85	76.65	36.25	70.00
BFCLv4	56.88	36.98	46.39	50.56	60.00
ToolSandbox	77.83	52.40	65.00	75.55	70.00
τ^3 -Bench Banking	5.67	3.35	4.12	5.45	5.00
Claw-Eval average (EN)	62.85	53.14	58.02	62.28	60.00
PinchBench	68.22	44.24	55.09	71.26	70.00
BrowseComp+ (OpenClaw)	26.89	8.31	15.90	24.46	20.00

LFM2.5-2.6B leads on every instruction-following benchmark and nearly every tool use benchmark, trailing only Qwen3.5-9B on BFCLv4. On agentic tasks, it outperforms both Gemma models across the board and trades closely with the Qwen models. On STEM, it leads on AA Omniscience and trails only Qwen3.5-9B on math. Coding is the one area where the larger models keep an edge.

These results make LFM2.5-2.6B a strong fit for high-volume agentic workloads on edge devices, especially when speed, privacy, and local deployment matter. For more complex agentic tasks or coding-heavy workloads, larger models may still be a better fit.¹

Fast Inference Everywhere

LFM2.5-2.6B ships with day-one support across the inference ecosystem:

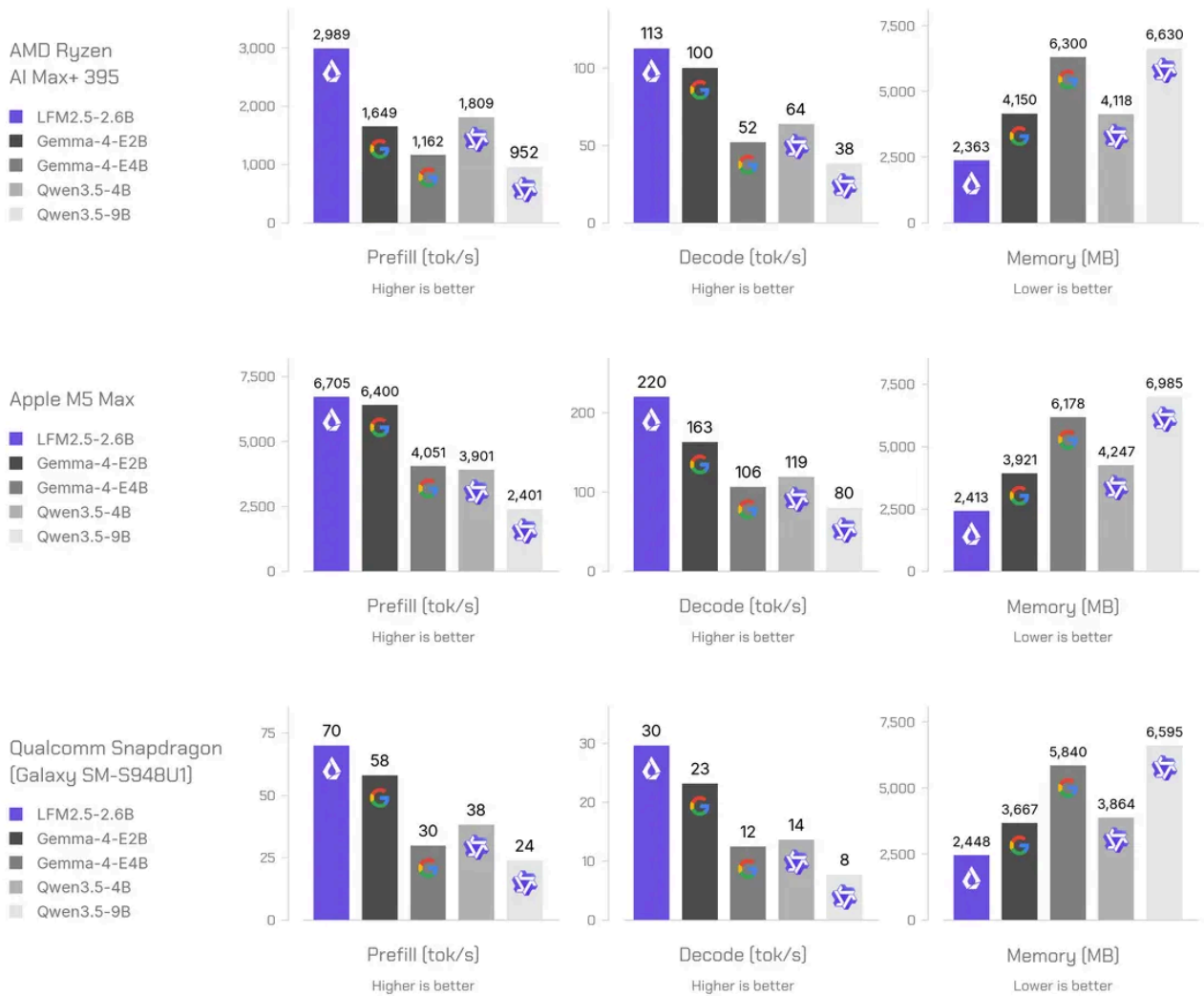
- **llama.cpp** – GGUF checkpoints for efficient edge inference
- **MLX** – Optimized inference for Apple Silicon
- **vLLM** – GPU-accelerated serving for production throughput
- **SGLang** – GPU-accelerated serving for production throughput
- **ONNX** – Cross-platform inference across diverse accelerators

CPU inference. Due to the efficient LFM2 architecture, LFM2.5-2.6B is the fastest model we tested at reading in prompts and generating answers, decoding 220 tokens/s on an M5 Max and 113 tokens/s on a Ryzen AI Max+ 395 while staying under 2.5 GB. It even holds 30 tokens/s on a phone, so a capable agent runs instantly and privately on your own device.

LFM2.5-2.6B

CPU Inference Metrics by Device

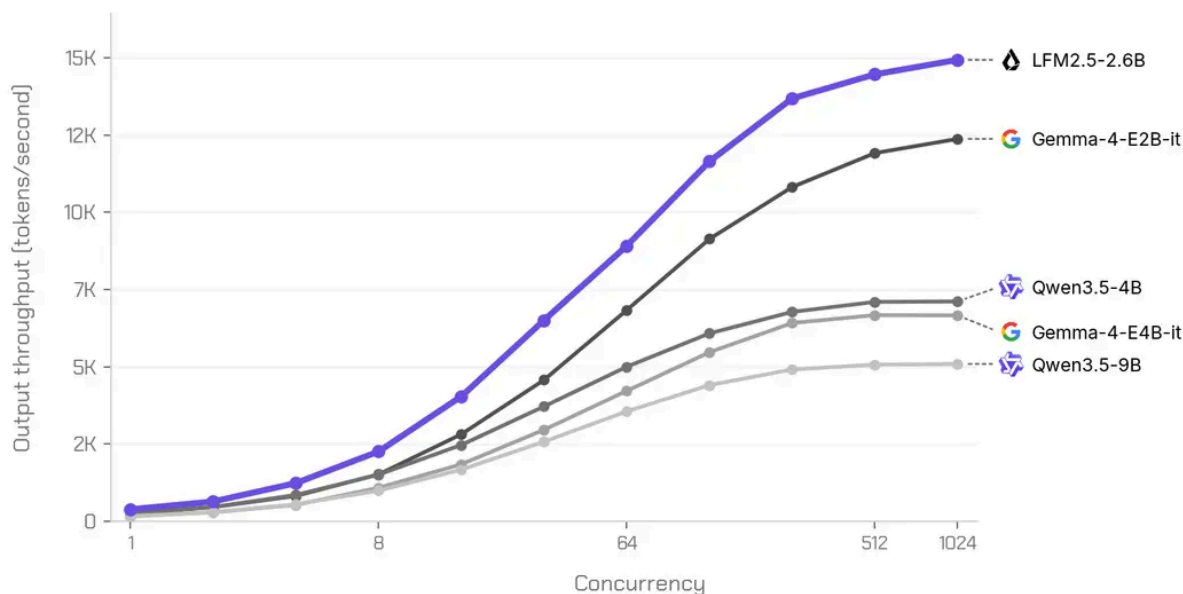
Q4_K_M quantization and 4K-token input context



GPU inference. We also measure output throughput (total output tokens / wall time) on a single NVIDIA H100 SXM5 GPU using a sustained-load setting: at each concurrency level, we continuously maintain the target number of in-flight requests, replacing each completed request immediately.

End-to-end throughput on 1x H100 GPU Inference Throughput by Concurrency

SGLang 0.5.16 | BF16 | D(1024,256) | 3 runs per level



We benchmark each model with SGLang 0.5.16, 1,024 input tokens, up to 256 output tokens, in BF16, averaging 3 runs per concurrency level. LFM2.5-2.6B is the fastest model in its size class, reaching almost 15K output tokens per second at high concurrency, roughly 1.3B tokens per day on a single H100.

Run local agents with LFM2.5-2.6B

LFM2.5-2.6B's size, speed, and capabilities make it a great choice for high-volume agentic workloads on edge devices. In the demo below, we run it inside the Liquid Agent harness on a phone, where it plans, calls tools, and works through a real task entirely on-device, without any cloud API calls.

Setting up your own local agent takes only two steps. First, serve LFM2.5-2.6B behind an OpenAI-compatible endpoint, then point your agent harness at it. It works out of the box with popular harnesses like [Hermes Agent](#), [OpenClaw](#), and [Pi](#). Check out [our guide](#) for how to serve the model locally and connect it with the agent harness of your choice.

Get Started

Start building today with LFM2.5-2.6B and LFM2.5-2.6B-Base, available on [Hugging Face](#).

With LFM2.5, we're delivering on our vision of AI that runs anywhere. These models are:

- **Open-weight** – Download, fine-tune, and deploy without restrictions

- **Fast from day one** – Native support for llama.cpp, MLX, and vLLM across Apple, AMD, Qualcomm, and NVIDIA hardware
- **A complete family** – From base models for customization to specialized audio and vision variants, one architecture covers diverse use cases

We can't wait to see what you build.



Download on Hugging Face



Read our docs



Try on Playground



Citation

Please cite this article as:

Liquid AI, "LFM2.5-2.6B: Deploy Agents Everywhere", Liquid AI Blog, Aug 2026.

```
@article{liquidAI202626B,  
  author = {Liquid AI},  
  title = {LFM2.5-2.6B: Deploy Agents Everywhere},  
  journal = {Liquid AI Blog},  
  year = {2026},  
  note = {www.liquid.ai/blog/lfm2-5-2-6b},  
}
```



¹ All the models were evaluated with vLLM and the following generation parameters:

- **BFCLv4**: temperature = 0.001, max output tokens = 4096.
- **ToolSandBox**: temperature = 0, max output tokens = 1024.
- **PinchBench**: temperature = 0.6, max output tokens = 8192.
- **τ^3 -Bench, Claw-Eval**: temperature = 0, no output limit. Qwen models use temperature = 0.6 (per the recommended settings), as greedy decoding degrades performance due to doom

looping.

- **Other evals:** temperature = 0.6, max output tokens = 32768.

Share on social:   

For press inquiries, email us at press@liquid.ai



Building the fastest, most compute-efficient,
and capable foundation models so
intelligence can live anywhere.

EST. 2023



Your email

Subscribe

MODELS

Liquid Foundation Models (LFMs)

Model Playground

Hugging Face

Apollo

DEVELOPER RESOURCES

Liquid Edge AI Platform (LEAP)

Demos

Documentation

Hackathons

Discord

GitHub

INDUSTRY SOLUTIONS

Automotive

Consumer electronics

E-commerce

Financial services

Healthcare

Industrial

Defense

RESEARCH

Liquid Labs

Automated Foundation Model Design

Blog

COMPANY

About

Careers

News

Events

FAQ

Contact

Shoutouts

